Neural ODE based modeling of memristive devices

Karol Bednarz, Bartłomiej Garda, Zbigniew Galias, *Senior Member, IEEE*

***Abstract*—The development of modern memristive technologies requires accurate dynamic models to enable reliable simulation and prototyping in applications such as resistive RAM, neural networks, and neuromorphic computing. Existing symbolic models often exhibit limitations in accurately capturing the complex dynamic behavior of physical devices across various operating conditions, impacting their reliability in simulations. This work proposes a neural-network-based modeling of memristor's behaviour using Neural Ordinary Differential Equation (Neural ODE) models. The proposed method employs the backpropagation technique based on solving the corresponding adjoint ODE backward in time using adjoint sensitivity methods. Experimental data obtained directly from Self-Directed Channel (SDC) memristors are used to train and validate the model. Results show that the Neural ODE approach more accurately reproduces the device's dynamic characteristics and provides improved flexibility and robustness compared to traditional theoretical models. Feasibility of Neural ODE–based frameworks for data-driven modeling of a broad class of physical memristive devices is demonstrated.**

***Index Terms*—Memristor modeling, artificial neural network, neural ODE.**

I. Introduction

NUMEROUS memristor models have been proposed following the discovery of memristive behavior at the nanoscale [1]. Physics-based approaches range from generalized device models capturing threshold-dependent switching [2], [3], [4] to compact representations of RRAM structures such as the Stanford model [5], flux-charge formulations for phase-change memory devices [6], and voltage-controlled threshold models suited for synaptic circuit design [7]. For Self-Directed Channel (SDC) memristors specifically, the Generalized Metastable Switch (GMS) model [8] and its derivatives—the Mean Metastable Switch (MMS) [9] and Generalized MMS (GMMS) [10]—provide semi-empirical formulations based on transition probabilities between metastable states. Memristive elements are also widely employed in chaotic systems, where they face similar modeling challenges, such as Hopfield neural networks designed to generate multi-scroll attractors through offset boosting techniques [11]. Despite their widespread adoption, these symbolic models share common limitations: they require extensive parameter tuning, employ window functions that introduce numerical discontinuities [12], [13], [14], and exhibit limited generalization when operating conditions deviate from calibration regimes [15], [16]. Furthermore, while physics-based models offer high accuracy, their mathematical complexity hinders large-scale circuit simulation, motivating efforts toward explicit reformulations [17]. Approximated linear models implemented in Verilog-A have also been proposed to reduce computational cost while preserving the ability to capture parasitic effects in crossbar memory arrays [18]. A recent attempt to address these issues using Physics-Informed Neural Networks [19] demonstrated the potential of data-driven approaches, but relied solely on synthetic data generated from existing theoretical models rather than physical device measurements.

In [20], the authors introduced Neural Ordinary Differential Equations (Neural ODEs), where hidden-state dynamics are governed by an ODE with a neural network defining the vector field. Training is performed end-to-end using adjoint sensitivity methods for efficient gradient computation.

This paper proposes two Neural ODE architectures for modeling memristive devices trained directly on experimental data from SDC memristors. The first, **SingleNN-MemODE**, uses a physics-informed memductance with a neural network modeling only the state dynamics. The second, **DualNN-MemODE**, employs neural networks for both memductance and state dynamics, enabling a higher-dimensional latent representation. We demonstrate that both models outperform the MMS and GMMS benchmarks in prediction accuracy while maintaining comparable or shorter training times.

The remaining part of this paper is organized as follows. In Sec. II, the proposed neural network architectures are presented along with the objective function and the reference models used for comparison. In Sec. III, the models are trained on experimental data from SDC memristors and the results are compared with existing methods. The last section concludes the study.

II. Modeling Memristors using Neural Networks

This study utilizes experimental data obtained from Self-Directed Channel (SDC) memristors doped with tungsten. The structure and properties of these devices have been described in detail in the literature, e.g., in [21].

The experimental setup, illustrated in Fig. 1, consists of an SDC memristor connected in series with a resistor. The sinusoidal input voltage is supplied by an arbitrary waveform generator. Voltage signals are measured using a data acquisition (DAQ) system. Measurements are carried out for the following combinations of supply voltage amplitudes and frequencies: $V_s \in \{0.5,\ 1.0,\ 1.5\}$ [V], $f \in \{1,\ 5,\ 10,\ 20,\ 50,\ 100\}$ [Hz].

The analyzed model of a memristor follows the general theoretical framework of memristive devices originally introduced in [22], and is expressed as

$$i(t) = \mathcal{G}\left(\mathbf{x}(t), v(t)\right) v(t), \tag{1a}$$

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = f\left(\mathbf{x}(t), v(t)\right), \tag{1b}$$

Manuscript received Month ??, 2026. This work was supported by the National Science Centre, Poland under Grant 2024/53/B/ST7/03841.

The authors are with the Department of Electrical Engineering, AGH University of Kraków, al. Mickiewicza 30, 30–059, Kraków, Poland, (e-mail: kbednarz@agh.edu.pl).

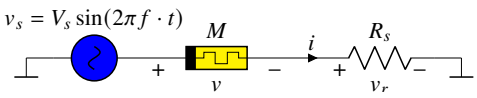

Fig. 1. Schematic diagram of the measurement setup used for the SDC memristor. The device is connected in series with a resistor, and the input voltage is applied via an arbitrary waveform generator. Voltage measurements are acquired using a data acquisition (DAQ) system.

where $\mathbf{x}$ denotes the vector of internal state variables, whose temporal evolution is governed by the function f, and $\mathcal{G}$ represents the memductance of the device. The fundamental challenge in memristor modeling lies in accurately identifying the functional forms of $\mathcal{G}(\mathbf{x}, v)$ and $f(\mathbf{x}, v)$.

A. Objective Function

The objective (loss) function $\mathcal{L}$ has four components

$$\mathcal{L} = \lambda_{\text{base}}\mathcal{L}_{\text{b}} + \lambda_{\text{vel}}\mathcal{L}_{\text{v}} + \lambda_{\text{curv}}\mathcal{L}_{\text{c}} + \sum_{i=1}^{N_c} \lambda_{\text{con}_i}\mathcal{L}_{\text{con}}(x_i). \quad (2)$$

The terms $\mathcal{L}_{\text{b}}$, $\mathcal{L}_{\text{v}}$, and $\mathcal{L}_{\text{c}}$ represent the NMSE (*Normalized Mean Squared Error*) of signal values, first derivatives, and second derivatives, respectively, between predicted and measured trajectories. The terms $\mathcal{L}_{\text{con}}(x_i)$ enforces physical constraints. The weighting coefficients are computed at each step using the Geometric Loss Strategy (GLS) [23].

Let $z_k = (z_{k,n})_{n=1}^{N}$ and $\tilde{z}_k = (\tilde{z}_{k,n})_{n=1}^{N}$ denote measured and predicted sequences for variable $z \in \{v, i, \dot{v}, \dot{i}, \ddot{v}, \ddot{i}\}$, where $k = 1, \ldots, M$ is the trajectory index and N is the number of samples. The normalized mean squared error is

$$\text{NMSE}(z, \tilde{z}) = \frac{1}{M}\sum_{k=1}^{M}\frac{1}{N}\sum_{n=1}^{N}\left(\frac{z_{k,n}}{\sigma_{z,k}} - \frac{\tilde{z}_{k,n}}{\sigma_{z,k}}\right)^2, \quad (3)$$

where normalization by the per-trajectory standard deviation $\sigma_{z,k}$ prevents amplitude disparities from biasing the optimization. The components of the objective function $\mathcal{L}$ are

$$\mathcal{L}_{\text{b}} = \text{NMSE}(v, \tilde{v}) + \text{NMSE}(i, \tilde{i}), \quad (4a)$$

$$\mathcal{L}_{\text{v}} = \text{NMSE}(\dot{v}, \tilde{\dot{v}}) + \text{NMSE}(\dot{i}, \tilde{\dot{i}}), \quad (4b)$$

$$\mathcal{L}_{\text{c}} = \text{NMSE}(\ddot{v}, \tilde{\ddot{v}}) + \text{NMSE}(\ddot{i}, \tilde{\ddot{i}}). \quad (4c)$$

The loss term for the inequality constraint $x > a$ is

$$\mathcal{L}_{\text{con}}(x) = \mathbb{E}\left[\sigma\left((a - x)\gamma\right)(a - x)^2\right], \quad (5)$$

where $\mathbb{E}[\cdot]$ denotes the ensemble average. The smooth sigmoid transition $\sigma(z) = (1 + e^{-z})^{-1}$ is used to maintain differentiability. The parameter $\gamma = 400$ controls the transition steepness.

B. Neural ODE based Memristor Modeling

In this study, two neural network-based modeling approaches are proposed called. The first one, denoted as **SingleNN-MemODE**, employs a deterministic formulation of the memristor memductance $\mathcal{G}$, while the function f defining the dynamics of internal variables is modeled using a neural network. In the second one, referred to as **DualNN-MemODE**, both $\mathcal{G}$ and f are modeled using neural networks. The conceptual architecture of the proposed approaches is illustrated in Fig. 2.

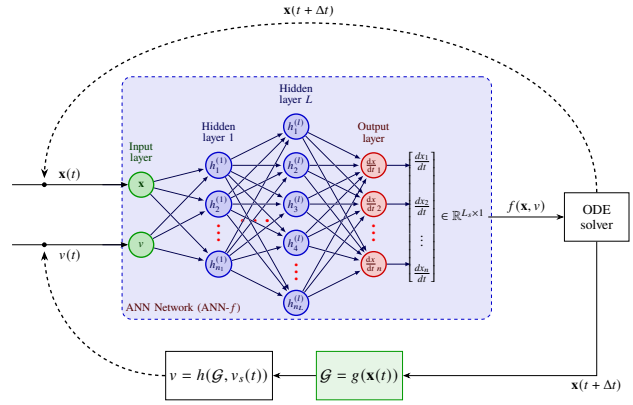

Fig. 2. The MemODE approach. The artificial neural network (ANN) evaluates the function $f(x, v)$, which is then integrated by the ODE solver to predict the state at the next time step $x(t + \Delta t)$. The upper dashed feedback loop indicates the recurrent nature of the process, where the output state is fed back as input for subsequent predictions. The memductance $\mathcal{G}$ is defined as the function g which is a deterministic function based on the internal variable x (SingleNN-MemODE) or the neural network with the input $\mathbf{x}(t)$ (DualNN-MemODE). The function $h(\mathcal{G}, v_s(t))$ computes the memristor voltage v given the memductance $\mathcal{G}$ and supply voltage $v_s(t)$.

For the SingleNN-MemODE approach, the function f is implemented using an artificial neural network with a single output. For the DualNN-MemODE approach the number of outputs is equal to the number of internal variables and may be larger than 1.

In the SingleNN-MemODE approach, the memductance is defined on the basis of the physical mechanisms underlying self-directed channel (SDC) devices, which is expressed as a function of the internal state variable x as follows:

$$\mathcal{G}(x) = x/R_{\text{ON}} + (1 - x)/R_{\text{OFF}}, \quad (6)$$

where R_{ON} and R_{OFF} denote the resistances in the low-resistance and high-resistance states, respectively. During the optimization process, the network parameters (weights $w_{k,i}$ and biases b_k) are tuned jointly with the values of R_{ON} and R_{OFF}. For this approach five inequality constraints are applied:

- $500\,\Omega < R_{\text{ON}} < R_{\text{OFF}} < 200\,\text{k}\Omega$,
- $0 < x(t) < 1$.

Bound for R_{ON} and R_{OFF} are based on measurements. Bounds for x(t) are consequences of the selected model.

In the DualNN-MemODEapproach, the memductance is modeled using the second neural network. This neural network takes the current state $\mathbf{x}$ as the input and produces the corresponding memductance $\mathcal{G}(\mathbf{x})$ as the output. The memristor state $\mathbf{x}$ is not constrained to be one-dimensional, enabling a more flexible representation of the device's internal dynamics. In this approach, the state variables are not restricted to any specific range, and hence only two measurement-based inequality constraint are used:

- $5\,\mu\text{S} < \mathcal{G}(\mathbf{x}) < 2\,\text{mS}$.

The learning process involves iteratively adjusting the network parameters to minimize the loss function that quantifies the difference between predicted and target outputs. This is achieved through backpropagation algorithm combined with the gradient descent optimization, where gradients of the loss

function with respect to each parameter are computed and used to update the weights and biases in the direction that reduces the overall error.

C. Neural Network Architecture

The neural network training is implemented using the `JAX` library [24] for array-oriented numerical computation and `Equinox` [25] for neural network construction in `Python`. The parameter gradients of the ODE solution are computed using adjoint sensitivity methods [20]. The `tsit5` adaptive Runge-Kutta solver is employed for its computational efficiency and numerical accuracy, with adaptive step-size control ensuring stable integration during training.

The neural network architecture, illustrated in Fig. 3, employs an expansion–compression (diamond-shaped) topology that incorporates a double reduction in dimensionality in both the initial and final hidden layers. This architectural configuration substantially decreases the overall number of trainable parameters and helps to mitigate vanishing and exploding gradient issues commonly encountered in deep architectures processing temporal sequences.

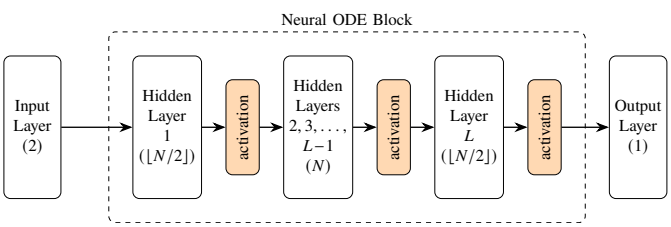

Fig. 3. Architecture of the feedforward neural network used within the Neural ODE framework for memristor dynamics modeling. The network consists of an input layer receiving two inputs, multiple hidden layers with decreasing-increasing-decreasing neuron counts $(\lfloor N/2 \rfloor, N, \ldots, N, \lfloor N/2 \rfloor)$, activation functions between layers, and a single output. The Neural ODE block encompasses the core computational layers that approximate the function $f(x(t), v(t))$ of the ODE system.

Several techniques are tested for the optimization of network weights. The choice of an optimizer and its parameters is guided by hyperparameter tuning conducted with the `Optuna` framework [26]. The hyperparameter search space comprises the number of hidden layers, the number of neurons per layer, and optimization settings (e.g. the batch size and the learning rate), with the aim of achieving a balance between model accuracy and generalization capabilities. The optimal configuration is selected based on minimizing the objective function on the validation set, ensuring that the model generalizes well to unseen data.

D. Existing Memristor Models for Comparison

The proposed models are compared with two established memristor models. The Mean Metastable Switch (MMS) model [9] describes the state dynamics as

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \frac{1}{\tau}\left(f_{\mathrm{ON}}(v)(1-x) - f_{\mathrm{OFF}}(v)x\right), \tag{7}$$

$$f_{\mathrm{ON}}(v) = \sigma\left(-\beta(v - V_{\mathrm{ON}})\right), \tag{8}$$

$$f_{\mathrm{OFF}}(v) = 1 - \sigma\left(\beta(v - V_{\mathrm{OFF}})\right), \tag{9}$$

where $\sigma(z) = (1 + e^{-z})^{-1}$, $\beta = q/(kT)$, and V_{ON}, V_{OFF} are the switching voltages. The memductance is defined by (6).

The Generalized MMS (GMMS) model [8], [10] extends MMS by modeling the memristor as a parallel connection of a memory-dependent element and a Schottky diode:

$$\begin{aligned} i(t) &= \eta\,\mathcal{G}(x)v + (1-\eta)\,i_{\mathrm{d}}(v), \\ i_{\mathrm{d}}(v) &= \alpha_{\mathrm{f}} e^{\beta_{\mathrm{f}} v} - \alpha_{\mathrm{r}} e^{-\beta_{\mathrm{r}} v}, \end{aligned} \tag{10}$$

where $\eta \in [0, 1]$ controls the ratio between memory-dependent and diode currents, and the state dynamics follow (7). The Schottky nonlinearity requires solving the DAE system $v_{\mathrm{s}} = R_{\mathrm{s}} i + v$ jointly with the state equation. Parameter estimation for both models employs global search using Adaptive Differential Evolution [27] followed by local L-BFGS-B refinement [28].

III. Results

During the hyperparameter tuning, a range of neural network architectures is systematically evaluated in order to identify the most suitable configuration for modeling memristor dynamics, as described in Sec. II-C.

In this study, the dataset consists of 18 trajectories obtained from measurements for all combinations of supply voltage amplitudes $V_{\mathrm{s}} \in \{0.5\,\mathrm{V}, 1.0\,\mathrm{V}, 1.5\,\mathrm{V}\}$ and frequencies $f \in \{1\,\mathrm{Hz}, 5\,\mathrm{Hz}, 10\,\mathrm{Hz}, 20\,\mathrm{Hz}, 50\,\mathrm{Hz}, 100\,\mathrm{Hz}\}$. The number of samples for each trajectory is 10^5 (100 periods of the supply voltage per trajectory, 1000 samples per period). This dataset is divided into the training dataset consisting of 14 trajectories and the validation datasets comprising 4 trajectories. Trajectories used for training and validation are selected to cover a diverse range of memristor's operating conditions, thereby enhancing the model's ability to generalize across different dynamic regimes. The validation dataset contains the following pairs of supply voltages and frequencies: $(1.5\,\mathrm{V}, 5\,\mathrm{Hz})$, $(1\,\mathrm{V}, 1\,\mathrm{Hz})$, $(1\,\mathrm{V}, 100\,\mathrm{Hz})$, and $(0.5\,\mathrm{V}, 5\,\mathrm{Hz})$.

The optimal SingleNN-MemODE configuration, achieved through hyperparameters optimization, comprises four hidden layers (64, 128, 128, 64 neurons) trained with `AdamW` (learning rate 1.89×10^{-2}, batch size 2) using the swish activation function. For DualNN-MemODE, the network computing $f(\mathbf{x}, v)$ has four hidden layers (120, 240, 240, 120 neurons) with 6 latent states, and the auxiliary network computing $\mathcal{G}$ has one hidden layer (80 neurons). Both are trained with `AdaBelief` (learning rate 6.30×10^{-3}, batch size 14), using GELU and ReLU activations, respectively.

A representative learning curve for SingleNN-MemODE is shown in Fig. 4. The consistent decrease of $\mathcal{L}$ on both training and validation sets over 360 epochs, along with their close agreement, confirms effective optimization and generalization. Representative simulation results demonstrating the neural network model's predictive capabilities are presented in Fig. 5 through comparative analysis of a tungsten-doped memristor device subjected to sinusoidal voltage excitation with an amplitude of 1.5 V and frequency of 100 Hz.

The effectiveness of the full loss function $\mathcal{L}$ defined in (2) was verified against using only $\mathcal{L}_{\mathrm{b}}$: for SingleNN-MemODE, the validation loss decreased from $\mathcal{L}_{\mathrm{b}} = 2.24 \times 10^{-2}$ to 1.85×10^{-2} when including $\mathcal{L}_{\mathrm{v}}$ and $\mathcal{L}_{\mathrm{c}}$.

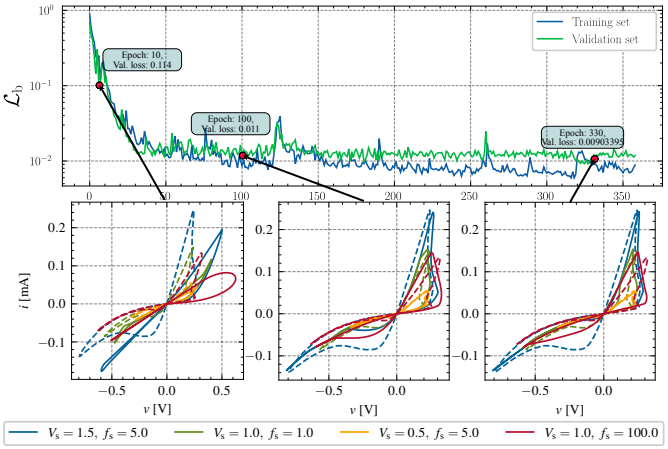

Fig. 4. Learning curve for the SingleNN-MemODE architecture over 360 epochs. Insets show v–i hysteresis loops at selected epochs (dashed: measured, solid: predicted).

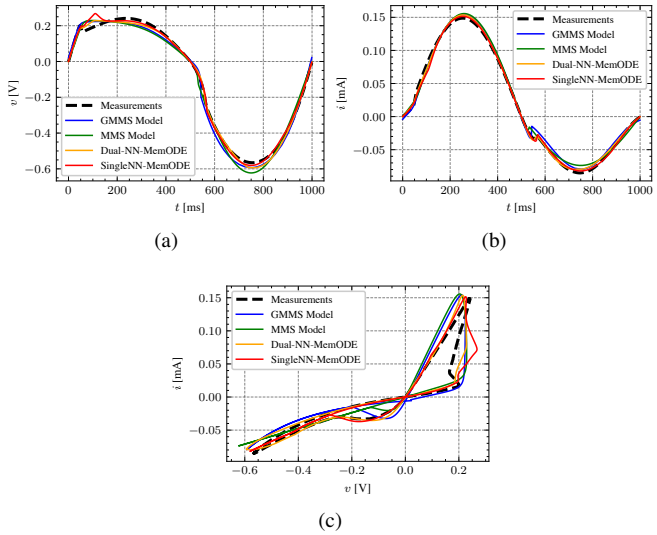

Fig. 5. Simulation test results for tungsten-doped memristor dynamics under sinusoidal voltage excitation (amplitude: 1.0 V, frequency: 1.0 Hz); (a) memristor's voltage waveform, (b) memristor's current response, (c) voltage-current hysteresis loop ($v - i$) illustrating the characteristic pinched behavior and memory properties fundamental to memristive operation.

A. Comparative Analysis with the MMS and GMMS Models

To assess the performance of the neural ODE approach, a comprehensive comparative analysis is conducted between the developed neural network models and the existing models of SDC memristors: the Mean Metastable Switch (MMS) model and the Generalized Mean Metastable Switch (GMMS) model.

This comparison is particularly significant as the MMS and GMMS models have been extensively validated against experimental data and serve as benchmarks for memristor circuit simulation [10], [29]. The comparative evaluation encompasses temporal voltage and current evolution, hysteretic characteristics, and quantitative loss function metrics to provide comprehensive assessment of modeling fidelity.

Table II reports the average loss function values $\mathcal{L}_b$ calculated for the validation, training and the whole datasets for the proposed and existing models. Results obtained for the validation dataset provide a quantitative evaluation of generalization performance of tested models across unseen memristor dynamics. The best result of 5.77×10^{-3} is obtained for the DualNN-MemODE model. The significant reduction of the average values of $\mathcal{L}_b$ for both proposed method when compared to existing methods demonstrates the enhanced modeling capability achieved through the neural ODE framework.

TABLE I
COMPARATIVE ANALYSIS OF THE LOSS FUNCTION $\mathcal{L}_b$. VALUES ARE REPORTED FOR DIFFERENT SUPPLY VOLTAGE AMPLITUDES V_s AND FREQUENCIES f_s. ROW-WISE MINIMUM AND MAXIMUM LOSS VALUES ARE COLOR-CODED IN GREEN AND RED, RESPECTIVELY. RECORDS BELONGING TO THE VALIDATION DATASET ARE MARKED BY BOUNDING BOXES.

Loss function values for V_s = 0.5 V				
f_s [Hz]	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
1	9.31×10^{-3}	1.22×10^{-3}	3.94×10^{-2}	2.43×10^{-2}
5	3.40×10^{-3}	6.43×10^{-3}	2.97×10^{-2}	1.46×10^{-2}
10	3.05×10^{-3}	2.13×10^{-3}	2.85×10^{-2}	2.60×10^{-2}
20	3.79×10^{-3}	1.02×10^{-3}	3.25×10^{-2}	1.14×10^{-1}
50	2.03×10^{-2}	5.60×10^{-3}	3.68×10^{-2}	3.45×10^{-2}
100	2.82×10^{-2}	1.07×10^{-2}	9.91×10^{-2}	1.11×10^{-1}
Loss function values for V_s = 1 V				
f_s [Hz]	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
1	5.28×10^{-3}	4.47×10^{-3}	1.30×10^{-2}	1.63×10^{-2}
5	8.25×10^{-3}	2.67×10^{-3}	1.46×10^{-2}	1.71×10^{-2}
10	4.72×10^{-3}	8.86×10^{-4}	1.49×10^{-2}	1.58×10^{-2}
20	7.77×10^{-3}	5.53×10^{-3}	1.88×10^{-2}	1.90×10^{-2}
50	3.29×10^{-2}	7.37×10^{-3}	6.29×10^{-2}	6.22×10^{-2}
100	4.48×10^{-2}	8.58×10^{-3}	4.53×10^{-2}	3.25×10^{-2}
Loss function values for V_s = 1.5 V				
f_s [Hz]	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
1	1.74×10^{-2}	2.48×10^{-3}	1.75×10^{-2}	4.74×10^{-2}
5	1.57×10^{-2}	3.60×10^{-3}	2.15×10^{-2}	3.71×10^{-2}
10	1.55×10^{-2}	3.70×10^{-3}	2.08×10^{-2}	2.31×10^{-2}
20	1.32×10^{-2}	7.93×10^{-3}	2.07×10^{-2}	2.41×10^{-2}
50	1.37×10^{-2}	8.41×10^{-3}	1.69×10^{-2}	4.70×10^{-2}
100	2.64×10^{-2}	1.20×10^{-2}	5.54×10^{-2}	2.41×10^{-2}

TABLE II
AVERAGE VALUES OF THE LOSS FUNCTION $\mathcal{L}_b$ FOR THE VALIDATION, TRAINING AND THE WHOLE DATASETS.

	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
validation	1.73×10^{-2}	5.77×10^{-3}	2.74×10^{-2}	2.51×10^{-2}
training	1.46×10^{-2}	5.11×10^{-3}	3.42×10^{-2}	4.22×10^{-2}
whole	1.52×10^{-2}	5.26×10^{-3}	3.27×10^{-2}	3.84×10^{-2}

Computation times are compared using a 3.5 GHz Intel i9-9900X processor. Training duration is 11.3 and 13.8 minutes for SingleNN-MemODE and DualNN-MemODE, respectively, compared to 20.4 minutes for MMS and 8.4 minutes for GMMS (using a two-stage global search followed by L-BFGS-B refinement [28]). However, trajectory generation (18 trajec-

tories) takes approximately 0.05 s for MMS/GMMS versus 4–6 s for the neural models (with relative tolerance 10^{-3}, and absolute tolerance 10^{-6}).

IV. Conclusions

This work presented two novel neural ODE-based architectures for modeling the dynamics of memristor devices. The proposed models leverage the power of neural networks to capture complex, nonlinear behaviors inherent in memristive systems, while maintaining the interpretability and physical consistency afforded by the ODE framework. The proposed models were applied for modeling of real Self-Directed Channel (SDC) memristors. Extensive evaluation against experimental data demonstrates that the neural ODE approach outperforms traditional phenomenological models, such as the MMS and GMMS models, in terms of accuracy and fidelity. The results highlight the capability of the neural ODE approach to learn intricate device dynamics directly from data.

Obtained results show that the computation time needed to optimize the proposed models is smaller than that for existing models. On the other hand the evaluation time needed to compute trajectories using the proposed models is significantly larger than for the MMS and GMMS models. The proposed models may also be used for modeling of memristors produced in other technologies. The findings of this study underscore the potential of neural ODEs as a versatile and effective framework for modeling complex physical systems, with applications in the design and analysis of memristor-based circuits. Developed methods pave the way to more accurate and efficient simulation tools, facilitating advancements in memristor technology and its integration into emerging computing paradigms, and open avenues for future research in data-driven modeling of nonlinear dynamical systems.

In future work, we plan to explore and use the proposed modelling framework for simulating the nonlinear dynamics of memristor-based chaotic circuits, and to investigate the potential of the proposed models for design and optimization of memristor-based neuromorphic systems.

References

[1] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, "The missing memristor found," *Nature*, vol. 453, no. 7191, pp. 80–83, May 2008.

[2] C. Yakopcic, T. M. Taha, G. Subramanyam, R. E. Pino, and S. Rogers, "A memristor device model," *IEEE Electron Device Letters*, vol. 32, no. 10, pp. 1436–1438, Oct. 2011.

[3] S. Kvatinsky, E. G. Friedman, A. Kolodny, and U. C. Weiser, "TEAM: ThrEshold Adaptive Memristor Model," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 60, no. 1, pp. 211–221, Jan. 2013.

[4] S. Kvatinsky, M. Ramadan, E. G. Friedman, and A. Kolodny, "VTEAM: A general model for voltage-controlled memristors," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 62, no. 8, pp. 786–790, 2015.

[5] X. Guan, S. Yu, and H.-S. P. Wong, "A SPICE compact model of metal oxide resistive switching memory with variations," *IEEE Electron Device Letters*, vol. 33, no. 10, pp. 1405–1407, Oct. 2012.

[6] J. Secco, F. Corinto, and A. Sebastian, "Flux–charge memristor model for phase change memory," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 65, no. 1, pp. 111–114, 2018.

[7] Y. Zhang, X. Wang, Y. Li, and E. G. Friedman, "Memristive model for synaptic circuits," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 64, no. 7, pp. 767–771, 2017.

[8] T. W. Molter and M. A. Nugent, "The generalized metastable switch memristor model," in *CNNA 2016; 15th International Workshop on Cellular Nanoscale Networks and their Applications*, 2016, pp. 1–2.

[9] L. Minati, L. Gambuzza, W. Thio, J. Sprott, and M. Frasca, "A chaotic circuit based on a physical memristor," *Chaos, Solitons & Fractals*, vol. 138, p. 109990, Sep. 2020.

[10] V. Ostrovskii, P. Fedoseev, Y. Bobrova, and D. Butusov, "Structural and parametric identification of Knowm memristors," *Nanomaterials*, vol. 12, no. 1, p. 63, Dec. 2021.

[11] Y. Li, C. Li, S. Zhang, Y. Zheng, and G. Chen, "Offset boosting-oriented construction of multi-scroll attractor via a memristor model," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 2, pp. 918–931, 2025.

[12] F. Corinto and A. Ascoli, "A boundary condition-based approach to the modeling of memristor nanostructures," *IEEE Trans. Circuits Syst. I*, vol. 59, no. 11, pp. 2713–2726, 2012.

[13] Z. Galias, "Simulations of memristors using the Poincaré map approach," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 3, pp. 963–971, Mar. 2020.

[14] V. A. Slipko and Y. V. Pershin, "Importance of the window function choice for the predictive modelling of memristors," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 68, no. 6, pp. 2167–2171, 2021.

[15] E. Linn, A. Siemon, R. Waser, and S. Menzel, "Applicability of well-established memristive models for simulations of resistive switching devices," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 61, no. 8, pp. 2402–2410, 2014.

[16] Z. Galias, "Study of amplitude control and dynamical behaviors of a memristive band pass filter circuit," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 65, no. 5, pp. 637–641, May 2018.

[17] V. Ntinas, A. Ascoli, I. Messaris, Y. Wang, V. Rana, S. Menzel, and R. Tetzlaff, "Toward simplified physics-based memristor modeling of valence change mechanism devices," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 5, pp. 2473–2477, 2022.

[18] N. Lupo, E. Pérez, C. Wenger, F. Maloberti, and E. Bonizzoni, "Analysis of parasitic effects in filamentary-switching memristive memories using an approximated verilog-a memristor model," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 66, no. 5, pp. 1935–1947, 2019.

[19] Y. Lee, K. Kim, and J. Lee, "A compact memristor model based on physics-informed neural networks," *Micromachines*, vol. 15, no. 2, 2024.

[20] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *32nd Conference on Neural Information Processing Systems (NeurIPS 2018)*, Montréal, Canada, 2018.

[21] K. A. Campbell, "Self-directed channel memristor for high temperature operation," *Microelectronics Journal*, vol. 59, pp. 10–14, Jan. 2017.

[22] L. Chua and S. M. Kang, "Memristive devices and systems," *Proceedings of the IEEE*, vol. 64, no. 2, pp. 209–223, 1976.

[23] R. Cipolla, Y. Gal, and A. Kendall, "Multi-task learning using uncertainty to weigh losses for scene geometry and semantics," in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Jun. 2018, pp. 7482–7491.

[24] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[25] P. Kidger and C. Garcia, "Equinox: neural networks in JAX via callable PyTrees and filtered transformations," *Differentiable Programming workshop at Neural Information Processing Systems 2021*, 2021.

[26] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD'19. ACM, Jul. 2019, pp. 2623–2631.

[27] R. Feldt, "Blackboxoptim.jl: Black-box optimization for Julia," 2013–2025. [Online]. Available: https://github.com/robertfeldt/BlackBoxOptim.jl

[28] C. Zhu, R. H. Byrd, P. Lu, and J. Nocedal, "Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization," *ACM Transactions on Mathematical Software*, vol. 23, no. 4, pp. 550–560, Dec. 1997.

[29] K. Bednarz and B. Garda, "Measurement and modeling of self-directed channel (SDC) memristors: An extensive study," *Energies*, vol. 17, no. 21, p. 5400, Oct. 2024.